

Make the concurrent forward progress guarantee usable in bulk

Document #: P3564R0
Date: 2025-01-13
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>
Bryce Adelstein Lelbach
<brycelelbach@gmail.com>
Michael Garland
<mgarland@nvidia.com>

Contents

- [1 Authors](#)
- [2 Introduction](#)
- [3 Why saying that agents are distinct fixes bulk’s forward progress](#)
 - [3.1 Summary](#)
 - [3.2 Execution agents relate to synchronization](#)
 - [3.3 Execution agents are more general than threads](#)
 - [3.4 This explains what it means for a single-threaded execution resource to have a forward progress guarantee](#)
- [4 Concurrent forward progress is not currently usable in bulk](#)
 - [4.1 Summary](#)
 - [4.2 Why we want bulk to have the strongest possible forward progress guarantee](#)
 - [4.2.1 Users need the freedom to express their intent](#)
 - [4.2.2 SG1 wants a bulk interface that “creates an execution agent per iteration”](#)
 - [4.2.3 “Run on N execution agents” != “parallelize this loop”](#)
 - [4.2.4 Parallel programming models give users a way to ask for concurrent forward progress where possible](#)
 - [4.3 Summary of design history for bulk execution’s forward progress](#)

- 5 Proposed solution
 - 5.1 Specify bulk as running on N distinct execution agents
 - 5.1.1 Summary
 - 5.1.2 Alternative designs
 - 5.2 Make it ill-formed to use default bulk with a scheduler promising concurrent forward progress
 - 5.3 Permit a bulk customization to fail if it cannot fulfill its forward progress for a given number of agents
 - 5.3.1 Summary
 - 5.3.2 Discussion
 - 5.3.3 Design implications
 - 5.3.4 Alternative designs
- 6 Implementation status
- 7 Wording
 - 7.1 Update version macro
 - 7.2 Change default bulk
 - 7.3 Specify behavior of bulk customizations
- 8 Appendix A: Design history for bulk execution’s forward progress
 - 8.1 Predecessors of P0443
 - 8.2 Earlier versions of P0443
 - 8.3 Later versions of P0443
 - 8.4 P2300
 - 8.5 How our proposal fits into this history
- 9 Appendix B: Chunked parallel for_each
- 10 References
- 11 Revision History

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

§ 2 Introduction

Scheduler-generic `std::execution` code can never assume concurrent forward progress in bulk’s function invocations, even if the scheduler’s `get_forward_progress_guarantee`

query returns concurrent. This is because a scheduler's forward progress guarantee relates to distinct execution agents on the scheduler, but nothing specifies when the different $f(k)$ invocations of the user's function f execute on distinct execution agents. Two function invocations that happen on the same agent cannot have a forward progress guarantee stronger than parallel, because they cannot perform blocking synchronization without the possibility of deadlock.

We propose fixing this by specifying that each of `bulk`'s function invocations happens in a distinct execution agent. This would make `bulk` more effective as a basis function for implementing asynchronous parallel algorithms. Since default `bulk` is sequential, we propose making it ill-formed to use default `bulk` with a scheduler that promises concurrent forward progress.

We will begin by explaining why we think saying that `bulk`'s function invocations happen in distinct execution agents implies that those function invocations have the scheduler's forward progress guarantee. Then, we will explain our interpretation of the current wording, and how it limits both users and implementers. Next, we will go through our proposed solution and contrast it with alternative designs. After that, we will present our proposed wording changes.

§ 3 Why saying that agents are distinct fixes `bulk`'s forward progress

§ 3.1 Summary

We propose saying that each of `bulk`'s N function invocations happens in a distinct execution agent. This section explains why we think that suffices to ensure that `bulk`'s function invocations have the same forward progress guarantee as the scheduler.

Execution agents do not necessarily correspond to hardware or operating system features. “These operations happen on distinct execution agents on the scheduler” is just the language that the Standard gives us to express that “the scheduler's forward progress guarantee applies to these operations.” This establishes a context for us to speak of forward progress of `bulk`'s function invocations, even when `bulk` is executed on a single-threaded execution resource.

For a scheduler using a single-threaded resource, `bulk`'s function invocations would still execute on distinct execution agents, but all those agents would run on the same thread. The agents would not necessarily ever be reified as an actual hardware or software entity; they are just a way to talk about forward progress of operations on the scheduler. A single-threaded scheduler like this could promise no stronger than parallel forward progress.

§ 3.2 Execution agents relate to synchronization

The Standard defines “execution agent” in the context of synchronization. [thread.req.lockable.general] 1 defines *execution agent* as “an entity such as a thread that may perform work in parallel with other execution agents.” Note 1 points out that “[i]mplementations or users can introduce other kinds of agents such as processes or thread-pool tasks.” The section goes on to explain that a “given execution agent” can “acquire or release ownership of a lock” ([thread.req.lockable.general] 3). The Standard generally uses “execution agent” when referring to locks and synchronization. For example, [thread.mutex.requirements.general] 1 says: “A mutex object facilitates protection against data races and allows safe synchronization of data between execution agents.” See also [thread.lock.general] 1 and [thread.sharedmutex.requirements.general] 2; the latter explains a situation in which “execution agents block” until some condition is met.

We therefore read other uses of “execution agent” in the Standard as relating to synchronization. For example, coroutines can be resumed on an execution agent ([coroutine.handle.resumption] 1); Note 2 explains that “[a] concurrent resumption of the coroutine can result in a data race.” This informs our reading of [exec.get.fwd.progress] 1, 3, which explains that the result of querying `get_forward_progress_guarantee` on a scheduler describes forward progress of distinct execution agents created by the scheduler.

§ 3.3 Execution agents are more general than threads

The phrase “execution agent” differs from *thread of execution* ([intro.multithread.general] 1), “also known as a *thread*,” which “is a single flow of control within a program, including the initial invocation of a specific top-level function, and recursively including every function invocation subsequently executed by the thread.” Execution agents are more general. For example, a program could use a SIMD (Single Instruction Multiple Data) lane as an “execution agent,” even though multiple SIMD lanes may share a single instruction stream (“flow of control”). This is, in fact, one of the motivations for the weakly parallel forward progress guarantee, as explained in the 2014 paper [N4156](#), “Light-Weight Execution Agents.”

§ 3.4 This explains what it means for a single-threaded execution resource to have a forward progress guarantee

This interpretation of “execution agents” lets us speak of “forward progress” even for a single-threaded execution resource. A scheduler on a single-threaded resource would execute bulk as a sequential for loop (for example, using default bulk). If we talk about each loop iteration as a distinct execution agent, then we could say that this bulk implementation promises at most parallel forward progress. That is, this scheduler’s bulk could deadlock if any iteration blocks on any other iteration.

In general, execution agents do not necessarily correspond to hardware or operating system features. “These function invocations happen on distinct execution agents on the scheduler” is just a way to say “the scheduler’s forward progress guarantee applies to

these function invocations.” This is the context for us to talk about forward progress of bulk.

§ 4 Concurrent forward progress is not currently usable in bulk

§ 4.1 Summary

Scheduler-generic code cannot assume that bulk’s N function invocations have a forward progress guarantee stronger than parallel, even if a scheduler’s `get_forward_progress_guarantee` query returns concurrent. The result of querying `get_forward_progress_guarantee` on the scheduler describes forward progress of distinct execution agents created by the scheduler (`[exec.get.fwd.progress]` 1, 3). The default bulk runs sequentially on a single execution agent (`[exec.bulk]` 4) and permits any number N of function invocations. As a result, even if the scheduler promises concurrent forward progress, default bulk cannot have forward progress stronger than parallel. For a customization of bulk, the current Working Draft does not specify how many agents it uses, just that those agents have the scheduler’s forward progress guarantee. For example, if a scheduler promises concurrent forward progress, but its custom bulk “tiles” the N function invocations across fewer than N execution agents, each invocation of bulk could promise at most parallel forward progress.

§ 4.2 Why we want bulk to have the strongest possible forward progress guarantee

§ 4.2.1 Users need the freedom to express their intent

Users want to write various algorithms. Some of those will require blocking synchronization between different activities, such as function invocations in bulk. Right now, they have no way of doing that in scheduler-generic code, except by launching successive back-to-back bulk operations. We want to make it possible for them to use the synchronization of their choice, if the scheduler supports it.

One motivation for permitting synchronization in bulk is to reduce algorithmic complexity. For example, bulk execution on some platforms has a launch cost that is independent of the number of execution agents. If we take away bulk and make users express execution over N agents via recursive calls to `when_all`, we force all concurrent execution into a $\log N$ launch cost. If we force users to launch multiple bulk operations, typical parallel algorithms with tree-shaped local communication patterns end up costing $\log N$ bulk launches instead of a constant number of launches.

§ 4.2.2 SG1 wants a bulk interface that “creates an execution agent per iteration”

SG1 polled with unanimous consent in its Wrocław 2024 review of [P3481R0](#), “[Summarizing std::execution::bulk\(\) issues](#),” that “[w]e need a version of the bulk API that creates an execution agent per iteration.”

§ 4.2.3 “Run on N execution agents” != “parallelize this loop”

A bulk that runs on N execution agents is a unique facility. Nothing else in Standard C++ lets users execute on N distinct execution agents “all at once.” We consider this a distinct capability from parallelizing a loop, and therefore deserving of a separate interface, for the following reasons.

1. Loop parallelization involves two separate tasks: mapping of loop iterations to execution agents, and executing work on each of those agents.
2. Users want, and popular programming models provide, more complicated mappings of iterations to agents than simple chunking. Users often want to *know* and even *specify* the mapping.

Regarding (1), one could build loop parallelization as a straightforward composition of the mapping (a function from agent index to a set of loop indices) with bulk (that executes a function of agent index on a distinct agent). Appendix B shows an example of how to implement a chunked parallel `for_each` algorithm in this way.

Regarding (2), examples of more complicated mapping facilities include the variety of `schedule` modifiers for OpenMP’s `for` clause, the many array mapping options in High Performance Fortran, and the generality of ScaLAPACK’s array distributions. Users need control over data distribution for best performance, so they can exploit knowledge of data locality relative to the various execution agents. Giving “implementation freedom” to bulk to hide loop distribution inside would take freedom away from users to build their own chunking schemes on top of bulk’s basic execution facility.

Regarding [P3481R0](#)’s proposed `bulk_chunked` interface: given an arbitrary mapping, it would be easier to implement loop parallelization with bulk than with `bulk_chunked`. For example, users might want to assign loop index k to agent $k \% N$ (out of N agents). As a result, each agent would only see contiguous “chunks” of size one.

§ 4.2.4 Parallel programming models give users a way to ask for concurrent forward progress where possible

Customizations of bulk should be able to promise concurrent forward progress, because popular parallel programming models try to provide both concurrent and parallel progress, and give users a way to ask for the one they want. Models offer users this choice in three different ways.

1. They expose different programming constructs for “running on N concurrent threads” versus “parallelizing a loop with M iterations over N threads” (analogous

to parallel `for_each`). Some models *only* have a way to run on N concurrent threads.

2. They expose a way to ask for nondefault “extra” forward progress guarantees, even though doing so might only have a performance benefit in special cases.
3. They expose a more complicated programming model, so that users can get concurrent forward progress between at least some execution agents.

Many parallel programming models expose different programming constructs for “launching N concurrent threads” versus “parallelizing a loop with M iterations over N threads.” The former lets users write algorithms that assume concurrent forward progress, while the latter makes the common case of distributing a large number of function invocations over a smaller number of execution agents easier.

- OpenMP¹ distinguishes parallel regions (e.g., `#pragma omp parallel`, with concurrent forward progress over N threads, with caveats; see below) from parallel loops (e.g., `#pragma omp parallel for`).
- MPI (the Message Passing Interface standard for distributed-memory parallel programming, first released in June 1994) *only* has a way to launch some number of processes (the distributed-memory analog of execution agents) with the equivalent of a concurrent forward progress guarantee. Users are expected to distribute parallel loop iterations over processes by hand.²
- PVM (the Parallel Virtual Machine): MPI’s ability to start a program with the desired number N of execution agents contrasts with another programming model of its day, PVM. PVM users who wanted N execution agents generally had to write boilerplate code that spawned agents in a binary tree (analogous to recursive `when_all` invocations), while MPI users simply got that from the beginning (after calling `MPI_Init` or the equivalent). This was a factor in MPI’s popularity over PVM.
- HPX includes both parallel algorithms like those in the C++ Standard Library, and different ways to enumerate execution agents of various kinds and execute work in bulk fashion with concurrent forward progress on them. For example, users can define a “parallel section” (analogous to a section of code executing on possibly multiple processes of an MPI communicator), and perform blocking synchronization across subsets of “images” (execution agents) in the parallel section. Users can launch their parallel section (via `define_spmc_block`) with a given number of images per “locality” (e.g., node of a cluster, or NUMA (Non-Uniform Memory Access) domain).

Some parallel programming models expose a way to ask for nondefault “extra” forward progress guarantees, even though doing so might only offer a performance benefit in special cases. CUDA³ distinguishes ordinary bulk execution (“device kernel”) launch (with at best parallel forward progress across thread blocks) from so-called “cooperative” bulk execution launch (with concurrent forward progress across thread blocks) as exposed

by [CUDA run-time functions](#) such as `cudaLaunchCooperativeKernel`. The cooperative launch feature is not “free”: it takes away the latency hiding feature of ordinary kernel launch, and thus only has performance benefits for specialized algorithms.

Finally, some programming models expose complication, so that users can get concurrent forward progress between at least some execution agents. This complication generally takes the form of a hierarchy, with different levels having possibly different forward progress guarantees. OpenMP 6.0 (released in November 2024) introduces “progress units” of some implementation-defined number of consecutive hardware threads. OpenMP’s parallel regions (`#pragma omp parallel`) offer concurrent forward progress across progress units, but only weakly parallel progress within a progress unit. Exposing this lets OpenMP support more architectures, at the cost of users needing to query the progress unit size (which might be one, indicating that all threads make concurrent forward progress). CUDA exposes two levels of parallelism: thread blocks and threads within a block. By default, thread blocks have the parallel guarantee, while threads within a block have the concurrent guarantee.⁴ CUDA provides various synchronization operations on groups of threads within a thread block, including synchronous barriers, that would only make sense given concurrent forward progress.

§ 4.3 Summary of design history for bulk execution’s forward progress

The “design intent” for such a thing as `std::execution` is not a single, static thought. Instead, it represents desiderata from a variety of coauthors from different institutions. These interests have been carried over through three major consolidations of proposals: the unification of three “executors” proposals that led to [P0443R0](#), the second unification (as expressed by [P1658](#) and [P1660](#)) that led to [P0443R11](#), and the third that led to replacing P0443 with [P2300](#). Nevertheless, one can observe a common set of principles from the various proposals and their reviews.

1. Bulk execution is a “basis operation,” that is, one of some minimal set of operations on a scheduler that could be used to construct asynchronous parallel algorithms.
2. Bulk execution may have different forward progress guarantees when running on different execution resources.
3. Bulk execution’s forward progress guarantees matter to users and should be defined.

Please see Appendix A for a review of this history.

§ 5 Proposed solution

§ 5.1 Specify bulk as running on N distinct execution agents

§ 5.1.1 Summary

1. `bulk` is not just another spelling of `for_each`, but a way to create execution agents in order to implement parallel algorithms. Therefore, `bulk` should offer the strongest possible forward progress guarantee, by being specified as each function invocation running in a distinct execution agent. As SG1 polled with unanimous consent in its Wrocław 2024 review of [P3481R0](#), “[w]e need a version of the `bulk` API that creates an execution agent per iteration.”
2. `bulk` should not have a different forward progress guarantee than any other operation on a scheduler, because otherwise it hinders reasoning about the forward progress guarantee of an operation defined by an arbitrary graph of senders.
3. Make it ill-formed to invoke default `bulk` with a custom scheduler that claims the concurrent forward progress guarantee. Make it ill-formed, no diagnostic required to invoke default `bulk` with a scheduler that claims the `parallel` forward progress guarantee, unless default `bulk` on that scheduler has that behavior.

§ 5.1.2 Alternative designs

We considered the following alternative designs.

1. Status quo: `bulk` uses an unspecified number of execution agents; those agents have their scheduler’s forward progress guarantee, but individual function invocations in `bulk` have at best parallel forward progress.
2. Let `bulk` take a minimum forward progress guarantee requirement as a parameter. `bulk` fails if it cannot satisfy the requirement.
3. Treat `bulk` as a special case, with a separate query for its per-invocation forward progress guarantee. (This was P0443’s design.)
4. Let each sender algorithm have its own scheduler-dependent forward progress guarantee, so that the `get_forward_progress_guarantee` query depends on the operation as well as the scheduler.
5. Require `bulk` to be customized, and require customizations to have the same forward progress guarantee as the scheduler’s.

We reject the status quo, Option (1), for reasons discussed above. Both default `bulk` and `bulk` customizations must provide parallel forward progress for their function invocations as long as their scheduler promises at least parallel forward progress. This is because `bulk` creates an asynchronous operation that performs its function invocations on a value completion operation ([`exec.bulk`] 6), and asynchronous operations on a scheduler “execute value completion operations on an execution agent belonging to the scheduler’s associated execution resource” ([`exec.async.ops`] 10). For example, it would not be legal for a scheduler’s `get_forward_progress_guarantee` query to return `parallel`, yet for `bulk` on that scheduler to use an OpenMP “`simd`” construct (which can only promise `weakly_parallel` forward progress).

None of Options (2), (3), and (4) would give users a way to know the mapping from function invocation to execution agent. Users need to know that for performance as well as correctness reasons. Given that loop distributions can be arbitrary one-to-one functions from $[0, N)$ to the set of execution agents, an interface for users to discover the implementation's mapping would be complicated. An interface for users to *specify* the mapping (analogous to OpenMP's various loop distribution options) would also be complicated, *and* would be separable from an interface for running on N execution agents. (OpenMP likewise separates running on N execution agents, `#pragma omp parallel`, from loop parallelization, `#pragma omp parallel for`.) The fundamental interface is "run on N execution agents."

Regarding Option (2), [P3481R0](#) proposes making `bulk` take an execution policy. This would have a similar effect, except that it would exclude concurrent forward progress.⁵ The effect of Option (2) is to make `bulk` like `parallel for_each`, in that it has a parameter analogous to an execution policy. However, if `bulk` is a basis operation – a way to implement parallel algorithms – and not just another way to spell `parallel for_each`, then developers would want to use `bulk` in a different way. When *using* parallel algorithms, execution policies like `par` and `par_unseq` communicate a "lower bound" forward progress requirement, often to expose opportunities for optimization. When *implementing* parallel algorithms, developers instead want to communicate the exact requirement of a particular code path. They might query the scheduler for its forward progress guarantee, and use it to dispatch to different code paths, as in the example below. This is because the parallel forward progress path might be less efficient than the concurrent forward progress path, for example because it replaces blocking synchronization with multiple `bulk` invocations.

```
constexpr auto g = get_forward_progress_guarantee(scheduler);
if constexpr (g == forward_progress_guarantee::concurrent) {
    // ... concurrent implementation ...
} else if constexpr (g == forward_progress_guarantee::parallel) {
    // ... parallel implementation ...
} else {
    static_assert(false, "Need at least parallel forward progress");
}
```

Regarding Option (4), it's clear to us what a forward progress guarantee means for `bulk`'s function invocations that complete on a single scheduler. However, it's not clear what that would mean for an arbitrary graph composed of various asynchronous operations that might complete on an arbitrary variety of senders. Thus, having the `get_forward_progress_guarantee` query depend on the sender algorithm would complicate the syntax without actually being useful for other sender algorithms besides `bulk`. This is why forward progress is a property of a scheduler.

Regarding Option (5), requiring `bulk` to be customized could be done either by removing the default implementation entirely, or by making it still selected for overload resolution but ill-formed. Users of `bulk` would either need to rely on a scheduler's customization, or find another scheduler that customizes `bulk` and then transition to that scheduler whenever

they use bulk. Either approach to removing default bulk would conflict with the P2300 design principle that every sender algorithm has a default. Sender algorithms are only customized for specific schedulers, and only because doing so is necessary for correctness or performance on that scheduler. If a scheduler has nothing to say about bulk, it should just leave the default in place. Removing default bulk would also have the unfortunate effect of bifurcating the set of schedulers into “bulk-capable” and “not-bulk-capable.” SG1 has polled to include bulk execution in the “minimal set” of an executor’s operations, for example in Kona 2017. In addition, removing default bulk would also prevent users from doing reasonable things like benchmarking default bulk against a customization.

Scheduler authors who want to promise concurrent forward progress but do not care about bulk can do either of the following.

- a. Nothing (do not customize bulk). This proposal will make it ill-formed to use default bulk with their scheduler (see the next section). The scheduler may thus ignore bulk when reporting its forward progress guarantee.
- b. Customize bulk, but have the customization always complete with an error if the number of agents N is greater than 1 (see the section after the next).

§ 5.2 Make it ill-formed to use default bulk with a scheduler promising concurrent forward progress

We propose that it be ill-formed to use default bulk with a scheduler whose `get_forward_progress_guarantee` query returns `concurrent`. This is because default bulk cannot possibly fulfill concurrent forward progress for any number of agents N other than one. This will prevent a common error for scheduler authors who do not care about bulk, yet still want to provide a concurrent scheduler. It also makes default bulk usable for users who don’t mind a weaker forward progress guarantee.

§ 5.3 Permit a bulk customization to fail if it cannot fulfill its forward progress for a given number of agents

§ 5.3.1 Summary

1. Permit a bulk customization to fail at run time if it cannot provide the promised forward progress guarantee for a given number of agents N .
2. This lets bulk still promise the strongest forward progress guarantee that it can provide for any value of N .
3. bulk fails by propagating a scheduler-dependent error completion.

§ 5.3.2 Discussion

Letting bulk fail if it cannot fulfill its forward progress guarantee for a given N would still permit bulk to promise the strongest forward progress guarantee that it can provide for

any value of `N`. By analogy, `std::thread` still promises an implementation-defined forward progress guarantee, even though `std::thread`'s constructor is permitted to fail. Just as `std::thread`'s constructor does not devolve to running code inline if it fails to launch a thread, `bulk` should not devolve to a weaker forward progress guarantee if it cannot meet its promise.

The idea that execution should “indicate failure” if it cannot provide the requested forward progress guarantee occurs in [P0058R1](#), one of the three predecessors of P0443 (see historical overview in Appendix A). SG1, in [its review of P0443R0 at the Issaquah 2016 meeting](#), expressed the wish for task launch to fail if it could not be launched with a concurrent progress guarantee.

Permitting `bulk` to fail for `N` “too large” makes this proposal trivial to implement. Scheduler authors who don’t care about `bulk` have two minimal-effort choices.

1. Do nothing. Default `bulk` will be ill-formed if their scheduler reports concurrent forward progress, and both well-formed and correct if their scheduler reports weaker forward progress than concurrent.
2. Customize `bulk` for their scheduler, and make `bulk` fail at run time for `N` greater than 1.

How should `bulk` fail? The idiomatic error reporting method for senders in `[exec]` is to use the error channel. The sender adapters `starts_on`, `schedule_from`, and `on` all handle failing to schedule an operation on a scheduler by executing an error completion on an unspecified execution agent. Why shouldn’t other operations that may require the creation of execution agents, like `bulk`, do the same? This behavior is analogous to that of `std::thread`'s constructor, which throws `std::system_error` “if unable to start the new thread” (`[thread.thread.constr]` 8).

What error should `bulk` report? We should not require the error to be an exception. This permits use of `std::execution` with custom schedulers in freestanding environments, or environments where exceptions are disabled. We should also let different schedulers report different error types. For example, a scheduler that creates execution agents one at a time using `jthread`'s constructor (not necessarily wise from a performance perspective, though it would guarantee concurrent forward progress) would reasonably just pass along the thrown `system_error` if thread creation fails. Forcing different schedulers to use the same error type would prevent custom schedulers from passing along information that might help users handle errors.

Implementing error handling by reporting down the error channel is practical in at least some cases. For example, OpenMP has a run-time function `omp_get_max_threads` that a `bulk` implementation could use to check if `N` is too large.

§ 5.3.3 Design implications

Customizations of `bulk` would need to know that they have N distinct execution agents available, before any of those agents may invoke the user's function. Any error reporting would need to happen before any function invocations. For example, it would be nonconforming for a customization to start some smaller number n of agents opportunistically, permit them to begin invoking the function for indices $[0, n)$, and only thereafter try creating more agents with the possibility of failure. In our view, this is an implication of the current specification; this proposal would not change that.

§ 5.3.4 Alternative designs

We do *not* accept any design permitting `bulk` to change its forward progress guarantee depending on N , for example by tiling function invocations across available execution agents if N is greater than the maximum number of concurrent agents. This would effectively make `bulk` a parallel loop interface. As we discuss elsewhere, we consider “running on N execution agents” separate functionality from “parallelize this loop,” and think the two should have distinct interfaces.

The alternative designs discussed below address the question of how users would prevent errors due to the number of agents N being too large. Different implementations' upper bounds for N may vary from 1 (OpenMP implementation running on a single-processor system that is not permitted to create threads) to numbers too large to fit in 32 bits (the maximum product of the x , y , and z dimensions of a grid of thread blocks in a CUDA kernel launch). Here are three designs. We do *not* propose any of them, but if we had to pick one, it would be the third design.

1. Let users query the scheduler's maximum number of execution agents that can execute with the promised forward progress guarantee: `size_t get_max_num_agents()`.
2. Add a variant of `bulk` for which users do not specify the number of execution agents N , but instead receive it as a second argument of their function.
3. Add a variant of `bulk` for which users specify a *maximum* number of execution agents N . They will then receive the *actual* number of execution agents n , where n is no larger than N , as the second argument of their function.

Regarding Option (1), we do not think a “maximum N ” query would be helpful for arbitrary user-defined schedulers. First, users may not *need* to run on the maximum number of agents, because they might only have a small number of tasks. Asking for more agents than needed might waste resources. Second, the query could only ever be an upper bound, because the actual maximum might change during the program's lifetime (as system resources are claimed by other processes or become available again) and might depend on the context (e.g., whether this is a nested `bulk` invocation, assuming that those are allowed).⁶ Third, the best upper bound for performance, or even a practically attainable upper bound, might be smaller than the maximum value. Just because `std::vector<T, Allocator>::max_size()` is huge doesn't necessarily mean that you can allocate that much memory on every system. Likewise, just because a scheduler *can* create

millions of concurrent execution agents, doesn't necessarily mean that this would perform acceptably.

To elaborate on (2) and (3), bulk's function would have two parameters `Shape k`, `Shape N`. The argument for `k` is the invocation index in $[0, N)$, and the argument for `N` is the actual number of execution agents.

Option (2) would correspond to using `#pragma omp parallel` without a `num_threads(N)` clause, where users can call `omp_get_num_threads` to query the number of threads that the scheduler offered. The bulk customization would provide some number `N` of execution agents (not necessarily the maximum – e.g., the scheduler may reduce the number to help with load balancing). Default bulk would provide `N=1`, thus ensuring that it has the same progress guarantee as any scheduler.

Option (2) has the same issue as Option (1), that users may only have a small number of function invocations to do; getting more agents than that might waste resources. Not giving users a way to specify `N` would force a branch `if (k < N) do_work(k)` into the user's function. Also, the optimal `N` value might depend on some properties of the user's function (e.g., its stack size requirement, or how many registers it uses) that the system might not be able to determine even at run time (e.g., if the function was loaded from a dynamically linked library). Thus, users still need a way to specify `N`.

Option (3) – letting users specify a maximum number of agents – would still force a branch in the user's function. However, it would expose an opportunity for users and the system to negotiate over the number of agents. Our one concern is that this would turn “failure to fulfill the forward progress guarantee” from an error into a non-error that is nevertheless the user's problem. Implementations could abuse this to claim concurrent forward progress while only ever offering `N = 1` execution agent. This is why we do not propose this feature, though we object to it the least of all three options here.

§ 6 Implementation status

Implementations would only need to make three changes to conform with this proposal.

1. Schedulers must only promise concurrent forward progress if they have a bulk customization and if that customization promises concurrent forward progress for its function invocations.
2. Default bulk must be ill-formed when used with a scheduler that promises concurrent forward progress.
3. If a scheduler has a bulk customization, and if the bulk customization can fail to provide the scheduler's forward progress guarantee for a given `Shape` argument `shape`, then the bulk customization must report an error (via `set_error`).

The custom schedulers in [NVIDIA's reference implementation of `std::execution`](#) already conform with this proposal, because they do not promise concurrent forward progress. For

example, its CPU-based `static_thread_pool::scheduler` reports parallel forward progress, because it uses a thread pool with a fixed number of threads. The GPU-based schedulers report weakly parallel forward progress. Thus, the only change we have left to implement would be (2).

§ 7 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The  character is used to denote a placeholder section number which the editor shall determine.

§ 7.1 Update version macro

In `[version.syn]`, change the value of the `__cpp_lib_senders` macro to YYYYMM, where the placeholder value YYYYMM denotes this proposal's date of adoption.

§ 7.2 Change default bulk

Change `[bulk.exec]` 4 as follows, so that it is ill-formed to complete default bulk if its completion scheduler promises concurrent forward progress. (The only changes to this section are in the body of the lambda assigned to `impls-for<bulk_t>::complete`.)

- 4 The member `impls-for<bulk_t>::complete` is initialized with a callable object equivalent to the following lambda:

```
[<class Index, class State, class Rcvr, class Tag, class... Args>
(Index, State& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
-> void requires see below {
    if constexpr (same_as<Tag, set_value_t>) {

        constexpr auto guarantee =
get_forward_progress_guarantee(decltype(get_completion_scheduler<set_value_t>
(get_env(rcvr))));
        static_assert(guarantee !=
forward_progress_guarantee::concurrent);

        auto& [shape, f] = state;
        constexpr bool nothrow = noexcept(f(auto(shape), args...));
        TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
            for (decltype(auto(shape)) i = 0; i < shape; ++i) {
                f(auto(i), args...);
            }
            Tag()(std::move(rcvr), std::forward<Args>(args)...);
        }());
    } else {
        Tag()(std::move(rcvr), std::forward<Args>(args)...);
    }
}
```

- 5 The expression in the *requires-clause* of the lambda above is `true` if and only if `Tag` denotes a type other than `set_value_t` or if the expression `f(auto(shape), args...)` is well-formed.

§ 7.3 Specify behavior of bulk customizations

Change `[bulk.exec]` 6 as follows, so that bulk customizations complete with an error completion if the customization is unable to execute on `shape` distinct execution agents. (The definition of the pack `args` has been moved from subparagraph 6.1 to paragraph 6. Subparagraph 6.1 has been rewritten and now includes two new subparagraphs 6.1.1 and 6.1.2.)

- 6 Let the subexpression `out_sndr` denote the result of the invocation `bulk(sndr, shape, f)` or an object equal to such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. Let `args` denote a pack of lvalue subexpressions referring to the value completion result datums of the input sender `sndr`. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation (`[exec.async.ops]`) that, when started,

(6.1) if `sndr` propagated a value completion operation, does exactly one of the following:

(6.1.1) — completes with an error completion operation with a scheduler-specific error on `rcvr`, if the asynchronous operation’s value completion operation would have been unable to invoke `f(i, args...)` on a distinct execution agent for every `i` of type `Shape` in `[0, shape)`; or

(6.1.2) — invokes `f(i, args...)` on a distinct execution agent for every `i` of type `Shape` in `[0, shape)`; and

(6.1) on a value completion operation, invokes `f(i, args...)` for every `i` of type `Shape` in `[0, shape)`, where `args` is a pack of lvalue subexpressions referring to the value completion result datums of the input sender, and

(6.2) propagates all completion operations sent by `sndr`.

§ 8 Appendix A: Design history for bulk execution’s forward progress

Appendix A reviews the history of `std::execution` in the context of bulk execution.

§ 8.1 Predecessors of P0443

P0443, the predecessor proposal of P2300, was itself a unification of three different “executors” proposals:

1. “Google’s executor model for interfacing with thread pools” N4414 (2015),

2. “Chris Kohlhoff’s executor model for the Networking TS” [N4370](#) (2015), and

3. “NVIDIA’s executor model for the Parallelism TS” [P0058](#).

Of these three proposals, only P0058 includes bulk execution. It defines different “tags” for specifying different levels of forward progress of bulk function invocations, including `concurrent_execution_tag`, as well as different kinds of executors (including `concurrent_executor`) that offer different levels of forward progress. It explains that different strengths of forward progress guarantee can be used to construct algorithms that communicate in different ways.

For example, parallel agents might communicate via a shared `atomic<T>` parameter, while concurrent agents might use a shared `barrier` object to synchronize their communication (Section 4.5).

`concurrent_execution_tag` - ... The basic idea is that the function invocations executed by a group of concurrent execution agents are permitted to block each others’ forward progress. This guarantee allows concurrent execution agents to communicate and synchronize (Section 4.10).

N4370 and its foundational proposals on asynchronous operations [N4045](#) (2014) and [N4242](#) (2014) do not speak of forward progress. N4414 makes forward progress a property of the specific executor – e.g., the `system_executor` has concurrent forward progress. However, it leaves a query for that property to future work.

An interface like that suggested around Execution Agents in N4156 (and related papers) could provide a generic mechanism for checking the traits of an executor (whether it is concurrent, parallel, or weakly parallel, as well as the behavior of thread local storage). Discussion of whether that concept should be applied to all executors is left for a subsequent paper but some form of execution agent traits is reasonable as [sic].

It may help to refer to [N4156](#) (“Light-Weight Execution Agents,” 2014), which defines “execution agent” in terms of the three levels of forward progress guarantees that later became part of the C++ Standard. It gives examples of implementations that provide different levels of forward progress for their execution agents.

§ 8.2 Earlier versions of P0443

SG1’s discussions of P0443 cover forward progress guarantees extensively, including guarantees of bulk execution. See e.g., SG1 discussion in [Issaquah 2016](#), [Kona 2017](#), and [Albuquerque 2017](#). Some reviewers mentioned wanting bulk execution to be able to provide concurrent forward progress if possible, for example if a bounded thread pool sometimes has enough threads for it. In Kona 2017, SG1 polled “Should the bulk two-way⁷ functions be part of the minimal set for a final TS?” with results 10/3/1/0/1.

[P0688R0](#) responded to Kona 2017 feedback on P0443R1. That feedback aimed to simplify P0443. P0668 does not clarify the forward progress of bulk execution’s individual function invocations; “blocking” for `bulk_*_execute` refers to the caller, as in, “[m]ay block forward progress of the caller until one or more invocations of `f` finish execution.” Minutes of SG1 discussion at the Toronto 2017 meeting (<https://wiki.edg.com/bin/view/Wg21toronto2017/P0688>) refer to the guarantees “within the bulk group” as distinct from the properties of the “spawning thread.” It also points out that whether an operation blocks the caller (in P0443, operations could be blocking or nonblocking; P2300 *only* has asynchronous operations) means something different than the forward progress guarantee of the operation.

§ 8.3 Later versions of P0443

Later versions of [P0443](#) introduced a customization point `bulk_execute` which is the analog and predecessor of P2300’s `bulk`. `bulk_execute` lets callers specify a separate forward progress guarantee for bulk execution, as a `bulk_guarantee` query. It has offered three different guarantees (unsequenced, sequenced, and parallel) since R4 of the proposal. The `parallel` guarantee means parallel forward progress, `sequenced` means a sequential loop, and `unsequenced` means something like `std::execution::par_unseq`.

The 2019 paper [P1658](#) led to the `bulk_execute` customization point design in later versions of P0443 (starting with R11). P1658 summarizes `bulk_execute` as “eagerly creat[ing] multiple execution agents in bulk.” [P1660R0](#) (“A Compromise Executor Design Sketch”) talks about different ways to express bulk execution – either eager, or lazy – but leaves the bulk execution interface as future work for P0443.

SG1’s Prague 2020 review of [P1897R2](#), “Towards C++23 executors: A proposal for an initial set of algorithms” (see [minutes](#)) reflects a debate on whether bulk execution is a fundamental building block. Much of this debate revolves around P0443’s separation of bulk executors and non-bulk executors. The final version [P1897R3](#) removes `indexed_for` (the “cleaned up version of `bulk_execute`”), with the suggestion that higher-level algorithms over ranges could be built atop bulk execution.

Discussion of [P1898](#), “Forward progress delegation for executors,” points out that forward progress guarantees are not as well-defined for asynchronous algorithms as they are for synchronous algorithms. This matters, for example, for launching nested work, like bulk iteration being launched inside bulk iteration.

We consider [P2181](#), “Correcting the Design of Bulk Execution” (R0 published June 2020, R1 published November 2020) to describe the last pre-P2300 state of affairs of bulk execution. It proposes changes to P0443R14 (which was the last published version of P0443).⁸ P0443R14 includes an “Editorial note” that says, “We should probably define what ‘execute N invocations of the function object F on the executor S in bulk’ means more carefully.” Section 4.1 of P2181 proposes defining that phrase in the following way.

A group of execution agents created in bulk has a *shape*. Execution agents within a group are identified by *indices*, whose unique values are the set of contiguous indices spanned by the group’s shape.

An executor *bulk executes* an expression by scheduling the creation of a group of execution agents on which the expression executes in bulk. Invocable expressions are invoked with each execution agent’s index. Bulk execution of expressions that are not invocables is executor-defined.

SG1 ended up giving feedback on P2181R1, but not on bulk’s forward progress. P2181 review stopped there, as the effort was folded into P2300.

§ 8.4 P2300

In P2300, the “Asynchronous inclusive scan” example that uses bulk (Section 1.3.2) says that it “creat[es]” or “spawn[s]” a number of “execution agents” equal to its input argument N. The example goes on to build an asynchronous inclusive scan algorithm out of a sequence of bulk and then operations. The example tiles the bulk operations by hand. However, the later nonwording description of bulk leaves unspecified the number of execution agents that bulk uses.

Each invocation of function runs in an execution agent whose forward progress guarantees are determined by the scheduler on which they are run. All agents created by a single use of bulk execute with the same guarantee. The number of execution agents used by bulk is not specified. This allows a scheduler to execute some invocations of the function in parallel.

The above text does not appear until R6 of P2300. R2 elaborates on the brief description in R0 and R1 with the following text that supports the “N execution agents” intent of bulk.

Each invocation of function runs in an execution agent whose forward progress guarantees are determined by the scheduler on which they are run. All agents created by a single use of bulk execute with the same guarantee. This allows, for instance, a scheduler to execute all invocations of the function in parallel.

The bulk operation is intended to be used at the point where the number of agents to be created is known and provided to bulk via its shape parameter.

A possible reason for the change in R6 is that bulk has always had a default sequential implementation since R0 of P2300. Making sender algorithms customizable and providing default implementations for them are key parts of P2300’s design, as explained in Section 5.4, “Sender algorithms are customizable.” However, bulk having a sequential default means that it cannot offer a concurrent forward progress guarantee, even if the scheduler does.

§ 8.5 How our proposal fits into this history

1. We want to restore [P2181](#)'s definition of bulk execution as running each function invocation on a distinct execution agent.
2. We also want to preserve default bulk's sequential implementation.
3. We propose resolving this apparent contradiction by using “execution agent” just as a way to talk about forward progress. This means that default bulk promises at best parallel forward progress.

§ 9 Appendix B: Chunked parallel `for_each`

This [Compiler Explorer example](#) shows how to build loop parallelization as a straightforward composition of the mapping (a function from agent index to a set of loop indices) with bulk (that executes a function of agent index on a distinct agent). We show the example's complete source code below.

```
#include <algorithm> // ranges::for_each
#include <cassert>
#include <functional> // invoke
#include <iostream>
#include <type_traits>
#include <ranges>
#include <vector>

namespace mystd {

template<class I, class Fun>
struct for_each_result {
    I in;
    Fun fun;
};

// Sequential implementation of something like ranges::for_each.
struct seq_for_each_fn {
    template<
        std::random_access_iterator I,
        std::sentinel_for<I> S,
        class Proj = std::identity,
        std::indirectly_unary_invocable<std::projected<I, Proj>> Fun
    >
    for_each_result<I, Fun>
    operator() (I first, S last, Fun f, Proj proj = {}) const {
        for (; first != last; ++first) {
            std::invoke(f, std::invoke(proj, *first));
        }
        return {std::move(first), std::move(f)};
    }
};
```

```

inline constexpr seq_for_each_fn seq_for_each;

// Given the index in [0, num_agents) of an agent,
// return the half-open interval of indices [first, last)
// in the range [0, num_elements)
// for which that agent is responsible.
//
// In case num_agents does not evenly divide num_elements,
// load-balance so that the number of indices per agent
// cannot vary by more than one.
template<std::integral Integral>
constexpr auto my_chunk(Integral num_elements, Integral
agent_index, Integral num_agents)
{
    constexpr Integral ZERO{0};
    constexpr Integral ONE{1};

    if (num_elements == 0) {
        return std::tuple{ZERO, ZERO};
    }

    auto quotient = num_elements / num_agents;
    // The first `remainder` agents get `quotient+1` number of
elements;
    // the rest of the agents get `quotient` number of elements.
    auto remainder = num_elements - quotient * num_agents;
    auto my_chunk_size = quotient + (agent_index < remainder ? ONE :
ZERO);

    // Cast back to Integral to undo effects of integer promotion;
    // otherwise, no matching call for std::min with Integral=short.
    auto num_incremented = std::min(
        agent_index, static_cast<Integral>(remainder));
    auto num_original = (agent_index >= remainder) ?
        (agent_index - remainder) : ZERO;

    auto my_chunk_index = num_incremented * (quotient + ONE) +
num_original * quotient;
    assert(
        my_chunk_index < num_elements ||
        (my_chunk_index == num_elements && my_chunk_size == ZERO)
    );

    // Cast back to Integral to undo effects of integer promotion
    // (for e.g., Integral=short).
    return std::tuple{
        static_cast<Integral>(my_chunk_index),
        static_cast<Integral>(my_chunk_index + my_chunk_size)
    };
}

// For the range [iterator, sentinel), return
// the given agent's [my_iterator, my_sentinel).
template<

```

```

        std::random_access_iterator I,
        std::sentinel_for<I> S
    >
    constexpr auto /* iterator, sentinel (not necessarily S) */
    my_chunk(I first, S last,
        std::iter_difference_t<I> agent_index,
        std::iter_difference_t<I> num_agents)
    {
        auto num_elements = std::ranges::distance(first, last);
        auto [my_first, my_last] = my_chunk(num_elements, agent_index,
num_agents);

        return std::tuple{first + my_first, first + my_last};
    }

    // Implementation corresponding to default bulk in [exec.bulk].
    template<std::integral Shape, std::invocable<Shape> Fun>
    void bulk(Shape N, Fun f) {
        for (Shape k = Shape(0); k < N; ++k) {
            std::invoke(f, k);
        }
    }

    // Chunked "parallel" implementation of something like
    ranges::for_each.
    // It uses the above "bulk" inside.
    struct chunked_for_each_fn {
        template<
            std::random_access_iterator I,
            std::sentinel_for<I> S,
            class Proj = std::identity,
            std::indirectly_unary_invocable<std::projected<I, Proj>> Fun
        >
        for_each_result<I, Fun>
        operator() (I first, S last, std::iter_difference_t<I>
num_agents, Fun f, Proj proj = {}) const {
            bulk(num_agents, [&] (std::iter_difference_t<I> agent_index) {
                auto [my_first, my_last] = my_chunk(first, last,
agent_index, num_agents);
                seq_for_each(my_first, my_last, f, proj);
            });

            return {std::move(first), std::move(f)};
        }
    };

    inline constexpr chunked_for_each_fn chunked_for_each;

} // namespace mystd

namespace test {

template<std::integral Integral>
void test_my_chunk_once(
    Integral num_elements, Integral num_agents)

```

```

{
    constexpr Integral ZERO{0};
    constexpr Integral ONE{1};

    Integral cur_first{0};
    Integral cur_last{0};
    for (Integral agent_index = ZERO; agent_index < num_agents;
        ++agent_index)
    {
        auto [first, last] = mystd::my_chunk(num_elements,
agent_index, num_agents);
        assert(first == cur_last);
        assert(last >= first);
        auto quotient = num_elements / num_agents;
        assert(last - first == quotient || last - first == quotient +
ONE);

        cur_first = first;
        cur_last = last;
    }
    assert(cur_last == num_elements);
    assert(cur_first <= cur_last);
}

template<std::integral Integral>
void test_my_chunk_tmpl() {
    constexpr Integral ZERO{0};
    constexpr Integral ONE{1};
    constexpr Integral TWO{2};
    constexpr Integral FIVE{5};
    constexpr Integral FIFTEEN{15};
    constexpr Integral SEVENTEEN{17};

    test_my_chunk_once(ZERO, ONE);
    test_my_chunk_once(ONE, ONE);
    test_my_chunk_once(ONE, SEVENTEEN);
    test_my_chunk_once(SEVENTEEN, ONE);
    test_my_chunk_once(SEVENTEEN, TWO);

    test_my_chunk_once(FIFTEEN, FIVE);
    test_my_chunk_once(SEVENTEEN, FIVE);
}

void test_my_chunk() {
    test_my_chunk_tmpl<short>();
    test_my_chunk_tmpl<unsigned short>();
    test_my_chunk_tmpl<int>();
    test_my_chunk_tmpl<unsigned int>();
    test_my_chunk_tmpl<long long>();
    test_my_chunk_tmpl<unsigned long long>();
}

} // namespace test

int main() {

```

```

test::test_my_chunk();

auto x = std::ranges::to<std::vector>(std::views::iota(1, 21));

std::cout << "std::ranges::for_each:\n";
[[maybe_unused]] auto result =
    std::ranges::for_each(x.begin(), x.end(), [] (float x_k) {
        std::cout << "x_k: " << x_k << "\n";
    });

std::cout << "\nseq_for_each:\n";
[[maybe_unused]] auto result_seq_0 =
    mystd::seq_for_each(x.begin(), x.end(), [] (float x_k) {
        std::cout << "x_k: " << x_k << "\n";
    });

std::cout << "\nseq_for_each with nontrivial projection:\n";
[[maybe_unused]] auto result_seq_1 =
    mystd::seq_for_each(x.begin(), x.end(), [] (float x_k) {
        std::cout << "10 * x_k: " << x_k << "\n";
    }, [] (float x_k) { return 10 * x_k; } );

int num_agents = 3;
std::cout << "\nchunked_for_each (num_agents=" << num_agents <<
"): \n";
[[maybe_unused]] auto result3 =
    mystd::chunked_for_each(x.begin(), x.end(), num_agents, []
(float x_k) {
    std::cout << "x_k: " << x_k << "\n";
    });

return 0;
}

```

§ 10 References

- L. S. Blackford et al., “ScaLAPACK Users’ Guide,” Society of Industrial and Applied Mathematics, Philadelphia, PA, 1997. Available online: <https://netlib.org/scalapack/slug/> [last accessed 2025/01/02].
- G. A. Geist, J. A. Kohl, and P. M. Papadopoulos, “PVM and MPI: a Comparison of Features,” *Calculateurs Paralleles*, Vol. 8 No. 2, 1996. Available online: <http://www.csm.ornl.gov/pvm/PVMvsMPI.ps> [last accessed 2024/12/31].
- Hartmut Haider et al., “HPX - The C++ Standard Library for Parallelism and Concurrency,” *Journal of Open Source Software*, September 2020. Available online: <https://joss.theoj.org/papers/10.21105/joss.02352> [last accessed 2025/01/10].
- HPX Documentation, v1.10.0. Available online: <https://hpx-docs.stellar-group.org/latest/pdf/HPX.pdf> [last accessed 2025/01/10].

- Ken Kennedy, Charles Koelbel, and Hans Zima, “The Rise and Fall of High Performance Fortran,” *Communications of the ACM*, Vol. 54, No. 11, pp. 74 - 82, November 2011.
- Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard Version 4.1”, November 2023. Available online: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf> [last accessed 2024/12/31].
- “OpenMP Application Programming Interface,” Version 6.0, November 2024. Available online: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf> [last accessed 2024/12/31].
- Parallel Virtual Machine website. Available online: https://www.csm.ornl.gov/pvm/pvm_home.html [last accessed 2024/12/31].

§ 11 Revision History

- Revision 0 (pre-Hagenberg) to be submitted 2025-01-13
-

1. OpenMP is a standard for thread-parallel programming, whose first version was released in 1997 (for Fortran; 1998 for C and C++).↩
2. Other parallel programming models contemporary to MPI 1.0 had built-in support for parallel data distributions and automatic data redistribution. MPI’s lack of these features simplified its implementation, and also forced users to write more specialized but therefore more performant code for explicit data movement. This performance difference helped MPI’s success relative to higher-level programming models such as High Performance Fortran. See Kennedy et al. 2011.↩
3. CUDA is a programming model that can be used for parallelism on NVIDIA’s graphics processing units (GPUs).↩
4. CUDA changed with the release of the Volta architecture, to support concurrent forward progress among threads within a thread block. This move from SIMT (Single Instruction Multiple Threads) thread scheduling to independent thread scheduling required both hardware changes and backwards-incompatible changes to users’ code.↩
5. Fixing that would require introducing a new Standard execution policy type, or defining a query on execution policies (including implementation-defined ones) for their forward progress guarantees.↩
6. An OpenMP-based implementation might have different maximum N values for bulk, based on whether the bulk is nested inside another bulk. This is because nested parallelism may make fewer threads available, so that OpenMP can keep its forward progress guarantee while preserving performance.↩

7. "One-way" interfaces are "fire and forget," with no proposed standard way to synchronize or get the result. "Two-way" interfaces return a future or have some other way to synchronize on the result of the asynchronous operation.↩
8. See also [P2224R0](#), "A Better `bulk_schedule`," which explains the design change between P2181R0 and P2181R1, as well as [minutes from SG1's summer 2020 discussion of P2181R0](#), and [minutes from SG1's December 16, 2020 discussion of P2224R0](#).↩